Building a Pitch Sequence Prediction Dataset with MLB Stats API

Section 1: Overview

This guide provides a comprehensive technical roadmap for creating a dataset to predict pitch sequences for specific pitcher-batter combinations using the MLB Stats API. The goal is to build a robust machine learning dataset that captures the complex dynamics of baseball pitch sequencing.

Key Data Requirements

To successfully predict pitch sequences, your dataset must include:

- **Pitch type and location data** - The actual pitches thrown and their spatial coordinates
- **Count information** - Balls, strikes, and outs in each situation
- **Game situation context** - Inning, score, runners on base, batting order
- **Pitcher characteristics** - Handedness, pitch arsenal, recent performance, seasonal statistics
- **Batter characteristics** - Handedness, performance metrics, tendencies against pitch types
- **Historical matchup data** - Previous encounters between specific pitcher-batter pairs
- **Sequence patterns** - Previous pitches in the at-bat and their outcomes
- **Pitcher expected statistics by pitch type** - xBA, xSLG, xWOBA per pitch type
- **Batter expected statistics vs pitch types** - Expected performance against different pitch types
- **Environmental conditions** - Temperature, humidity, weather conditions

Section 2: Primary API Endpoints

2.1 Live Game Feed (GUMBO) - Primary Data Source

Endpoint: `GET /api/v1.1/game/{game_pk}/feed/live`

This is your primary endpoint for comprehensive pitch-by-pitch data. The Live Game Feed provides the most detailed view of game events and is essential for sequence modeling.

Key Parameters:

- `game_pk`: Unique game identifier (required)
- `timecode`: Optional parameter to get data at a specific point in time

Returns comprehensive game data including:

- All plays with complete pitch-by-pitch details
- Play outcomes and event descriptions

- Pitch types, velocities, and coordinate data
- Count information (balls, strikes, outs) for each pitch
- Base/out situations and runner positioning
- Pitcher and batter IDs for every matchup
- Game context (inning, score, situation)
- Weather information (temperature, conditions, wind)

2.2 Play-by-Play Endpoint

Endpoint: `GET /api/v1/game/{game_pk}/playByPlay`

This endpoint provides an alternative, more structured view of game plays. While less detailed than the Live Game Feed, it offers a cleaner format for basic play sequencing analysis.

2.3 Analytics Endpoint

Endpoint: `GET /api/v1/game/{gamePk}/{guid}/analytics`

This endpoint contains advanced Statcast metrics and pitch tracking data when available. It's particularly valuable for enhanced pitch location and movement data that can improve model accuracy.

Section 3: Player Historical Stats

3.1 Standard Player Stats

Endpoint: `GET /api/v1/people/{personId}/stats`

Key Parameters:

- `stats`: "pitching" or "hitting"
- `season`: Year (e.g., "2025")
- `group`: "pitching" or "hitting"

This endpoint provides seasonal statistics that serve as important features for understanding player tendencies and performance levels.

3.2 Pitcher vs Batter Matchup Data (Critical)

Endpoint: `GET /api/v1/people/{personId}/stats/grouped`

Parameters:

- `stats`: ["VS_PLAYER", "VS_PLAYER_5Y", "VS_PLAYER_TOTAL"]
- `group`: ["pitching"] for pitcher perspective or ["hitting"] for batter perspective
- `season`: Specific year or use VS_PLAYER_5Y for 5-year historical data

Returns:

- Historical performance metrics for specific pitcher-batter matchups

- Essential for building features about prior encounters and tendencies
- Critical for understanding how pitchers approach familiar batters

3.3 Expected Statistics by Pitch Type

Endpoint: `GET /api/v1/people/{personId}/stats/grouped`

Parameters:

- `stats`: "EXPECTED_STATISTICS"
- `group`: "pitching" for pitcher perspective or "hitting" for batter perspective
- `season`: Specific year
- `groupBy`: "pitch_type"

Returns:
This endpoint provides Statcast expected statistics broken down by pitch type, which are crucial for understanding the true quality of contact and performance beyond traditional stats.

For Pitchers (pitches thrown):

- Expected Batting Average (xBA) allowed per pitch type
- Expected Slugging Percentage (xSLG) allowed per pitch type
- Expected Weighted On-Base Average (xWOBA) allowed per pitch type
- Data available for each pitch type in arsenal (FF, SL, CH, CU, SI, FC, etc.)

For Batters (pitches faced):

- Expected Batting Average (xBA) vs each pitch type
- Expected Slugging Percentage (xSLG) vs each pitch type
- Expected Weighted On-Base Average (xWOBA) vs each pitch type
- Performance expectations against different pitch categories

Quality of Contact Metrics:
These expected statistics use Statcast data to measure the quality of batted ball contact, providing a more predictive measure than traditional outcomes. They help identify:

- Which pitch types a pitcher executes most effectively
- Which pitch types a batter handles best/worst
- True talent level independent of luck and defense

3.4 Weather Data

Endpoint: `GET /api/v1.1/game/{game_pk}/feed/live`

Weather information is embedded within the main game feed and can significantly impact pitch movement and game strategy.

Data Location:
Navigate to `gameData.weather` in the response JSON.

Available Weather Fields:

- `temp`: Temperature in Fahrenheit
- `condition`: Weather condition (Clear, Cloudy, Overcast, Rain, etc.)
- `wind`: Wind speed and direction
- `humidity`: Humidity percentage (when available)

Additional Stadium Information:

- `gameData.venue`: Contains stadium information
- `gameData.venue.roof`: Indicates outdoor, indoor, or retractable roof
- Indoor stadiums (domes) will have null weather values

Data Quality Notes:

- Not all games have complete weather data
- Indoor stadiums typically have controlled conditions
- Historical games may have incomplete weather records
- Weather can change during games, but API typically shows game-time conditions

Section 4: Data Collection Strategy

Step 1: Identify Games to Collect

Use the schedule endpoint to systematically identify games for data collection:

Endpoint: `GET /api/v1/schedule`

Parameters:

- Date ranges for the seasons you want to analyze
- Filter by team, league, or specific date ranges
- Extract game_pk values for each game in your target dataset

Step 2: Extract Pitch-Level Data

For each game_pk, implement the following data extraction pattern:

```
# Pseudo-code structure for pitch data extraction
response = requests.get(f'https://statsapi.mlb.com/api/v1.1/game/{game_pk}/feed/live')
game_data = response.json()

# Extract weather data
weather = game_data['gameData'].get('weather', {})
temperature = weather.get('temp')
wind = weather.get('wind')
```

```
condition = weather.get('condition')
humidity = weather.get('humidity')

# Navigate to the plays section
plays = game_data['liveData']['plays']['allPlays']

for play in plays:
    matchup = play['matchup']
    pitcher_id = matchup['pitcher']['id']
    batter_id = matchup['batter']['id']

    # Extract each pitch in the at-bat
    for pitch in play['playEvents']:
        if pitch['isPitch']:
            pitch_data = {
                'game_pk': game_pk,
                'pitcher_id': pitcher_id,
                'batter_id': batter_id,
                'pitch_number': pitch['pitchNumber'],
                'pitch_type': pitch['details']['type']['code'],
                'pitch_type_desc': pitch['details']['type']['description'],
                'balls': pitch['count']['balls'],
                'strikes': pitch['count']['strikes'],
                'outs': pitch['count']['outs'],
                'pitch_speed': pitch['pitchData']['startSpeed'],
                'pitch_x': pitch['pitchData']['coordinates']['pX'],
                'pitch_z': pitch['pitchData']['coordinates']['pZ'],
                'plate_x': pitch['pitchData']['coordinates']['x'],
                'plate_z': pitch['pitchData']['coordinates']['y'],
                'outcome': pitch['details']['description'],
                'inning': play['about']['inning'],
                'runners_on': get_runners(play['matchup']),
                'score_diff': calculate_score_diff(game_data),
                'temperature': temperature,
                'humidity': humidity,
                'weather_condition': condition,
                'wind': wind,
            }
```

Step 3: Enrich with Player Characteristics

For each unique pitcher and batter identified in your dataset:

- Retrieve season statistics using `/api/v1/people/{personId}/stats`
- Extract pitch arsenal data (percentage of each pitch type used)
- Gather performance splits (vs LHP/RHP, home/away, by count, etc.)
- Collect recent performance metrics (last 5-10 games)

Step 3.5: Collect Expected Statistics by Pitch Type

For both pitchers and batters, collect expected statistics broken down by pitch type:

```
# Get pitcher expected stats by pitch type
expected_stats = requests.get(
    f'https://statsapi.mlb.com/api/v1/people/{pitcher_id}/stats/grouped',
    params={
        'stats': 'EXPECTED_STATISTICS',
        'group': 'pitching',
        'season': current_season,
        'groupBy': 'pitch_type'
    }
)
# Returns xBA, xSLG, xWOBA for each pitch type (FF, SL, CH, etc.)

# Get batter expected stats vs pitch types
batter_expected_stats = requests.get(
    f'https://statsapi.mlb.com/api/v1/people/{batter_id}/stats/grouped',
    params={
        'stats': 'EXPECTED_STATISTICS',
        'group': 'hitting',
        'season': current_season,
        'groupBy': 'pitch_type'
    }
)
# Returns expected performance vs each pitch type faced
```

Step 4: Add Historical Matchup Context

For each pitcher-batter pair, gather historical encounter data:

```
# Example of historical matchup data collection
matchup_stats = requests.get(
    f'https://statsapi.mlb.com/api/v1/people/{pitcher_id}/stats/grouped',
    params={
        'stats': 'VS_PLAYER_5Y',
        'group': 'pitching',
        'season': current_season
    }
)
# Extract: at-bats, hits, strikeouts, walks, home runs in previous encounters
```

Section 5: Key Data Fields for Modeling

Situational Features

Count State:

- Balls (0-3)
- Strikes (0-2)

- Outs (0-2)

Game Context:

- Inning (1-9+)
- Score differential (team batting - team pitching)
- Runners on base (binary indicators for 1st, 2nd, 3rd)
- Batting order position (1-9)
- Home/away indicator

Pitcher Features

Identity and Physical:

- Pitcher ID and handedness (L/R)
- Age and experience level

Performance Metrics:

- Season statistics (ERA, K/9, BB/9, WHIP, FIP)
- Recent performance (ERA and K/BB ratio over last 5 starts)
- Pitch usage patterns by count situation
- Historical success rate against this specific batter

Arsenal Characteristics:

- Percentage usage of each pitch type
- Average velocity by pitch type
- Pitch movement profiles
- Preferred sequences and patterns

Expected Statistics by Pitch Type:

- xBA allowed for each pitch type (FF, SL, CH, CU, etc.)
- xSLG allowed for each pitch type
- xWOBA allowed for each pitch type
- Quality of contact metrics per pitch in arsenal
- True effectiveness independent of defensive positioning

Batter Features

Identity and Physical:

- Batter ID and handedness (L/R/S)
- Age and experience level

Performance Metrics:

- Season statistics (AVG, OBP, SLG, wOBA, K%, BB%)
- Performance splits vs pitcher handedness
- Success rates against specific pitch types
- Recent performance trends
- Historical success against this specific pitcher

Approach Characteristics:

- Plate discipline metrics (swing rates, contact rates)
- Performance by count
- Situational hitting (RISP, high leverage)

Expected Statistics vs Pitch Types:

- xBA vs fastballs (FF, SI, FC)
- xBA vs breaking balls (SL, CU, KC)
- xBA vs off-speed pitches (CH, FS)
- xSLG and xWOBA vs each specific pitch type
- Performance against different pitch types based on contact quality

Environmental Features

Weather Conditions:

- Temperature (degrees Fahrenheit)
- Humidity percentage
- Wind conditions (speed and direction)
- Weather condition (Clear, Cloudy, Rain, Dome, etc.)

Stadium Characteristics:

- Stadium type (outdoor/indoor/retractable roof)
- Altitude and geographic factors
- Playing surface conditions

Impact on Performance:

- Temperature effects on ball flight and pitcher stamina
- Wind impact on pitch movement and carry
- Humidity effects on grip and ball behavior
- Indoor vs outdoor playing conditions

Sequence Features

Recent Pitch History:

- Previous 1-3 pitch types in the at-bat
- Previous 1-3 pitch locations
- Previous pitch outcomes (ball, strike, foul, in-play)
- Pitch number in the current at-bat

Pattern Recognition:

- Common sequences this pitcher uses
- Batter's success against similar sequences
- Count-specific tendencies
- Within-game adaptation patterns

Target Variable Options

Classification Targets:

- Next pitch type (multi-class: FF, SL, CH, CU, etc.)
- Next pitch location zone (strike zone grid)
- Pitch intent (strike/ball/waste pitch)

Regression Targets:

- Exact pitch coordinates (pX, pZ)
- Pitch velocity
- Spin rate (if available)

Section 6: Implementation Tips

Rate Limiting and API Etiquette

Best Practices:

- While MLB Stats API doesn't officially document rate limits, implement respectful request patterns
- Add 0.5-1 second delays between requests to avoid overwhelming the servers
- Cache API responses locally to minimize redundant calls
- Implement retry logic with exponential backoff for failed requests

Data Volume Considerations

Scale Planning:

- A full MLB season includes approximately 2,430 games (162 games × 30 teams ÷ 2)
- Average of ~300 pitches per game results in ~730,000 pitches per season
- Multi-season datasets can easily exceed 2-3 million pitch records
- Store in efficient formats: Parquet for analytics, PostgreSQL for relational queries

Data Quality Management

Common Issues:

- Not all games have complete Statcast data, especially for older seasons
- Missing pitch coordinate data should be handled gracefully
- Some pitch type classifications may be inconsistent between games
- Weather delays and suspended games can create data anomalies
- Expected statistics only available from 2015+ (Statcast era)
- Weather data may be missing for indoor stadiums or incomplete for older games
- Temperature typically ranges from 40-100°F for outdoor games
- Indoor stadiums will have null weather values

Mitigation Strategies:

- Implement data validation checks for critical fields
- Group similar pitch types when dealing with classification inconsistencies
- Create flags for incomplete or questionable data
- Maintain data lineage for debugging and quality assessment
- Handle missing expected statistics for pre-2015 data
- Use stadium type to determine if weather data should be expected

Handling the GUMBO Response Structure

Response Navigation:

- The Live Game Feed response is large and deeply nested
- Focus on `liveData.plays.allPlays` for pitch-by-pitch sequences
- Use `gameData` section for game context and metadata
- Reference `liveData.boxscore` for summary statistics and validation
- Extract weather data from `gameData.weather`

Section 7: Example Full Workflow

1. Get Game Schedule for Date Range

```
# Example API call for schedule
schedule_url = "https://statsapi.mlb.com/api/v1/schedule"
params = {
    'startDate': '2025-04-01',
    'endDate': '2025-10-01',
    'sportId': 1  # MLB
}
schedule_response = requests.get(schedule_url, params=params)
games = schedule_response.json()['dates']
```

2. Extract Game PKs and Fetch Live Feeds

```
game_pks = []
for date in games:
    for game in date['games']:
        if game['status']['statusCode'] == 'F':  # Final games only
            game_pks.append(game['gamePk'])

# Fetch live feed for each game
for game_pk in game_pks:
    live_feed_url = f"https://statsapi.mlb.com/api/v1.1/game/{game_pk}/feed/live"
    game_data = requests.get(live_feed_url).json()
    # Process pitch data as shown in Step 2 above
```

3. Parse Pitch Sequences from Nested JSON

Implement robust JSON parsing that handles missing fields and unexpected structures. Create a standardized pitch record format that captures all essential features for modeling.

4. Build Unique Player Lists

```
# Extract unique pitcher and batter IDs
unique_pitchers = set()
unique_batters = set()

for pitch_record in all_pitches:
    unique_pitchers.add(pitch_record['pitcher_id'])
    unique_batters.add(pitch_record['batter_id'])
```

5. Fetch Player Stats and Historical Matchups

```
def get_expected_stats_by_pitch(person_id, group, season):
    """Get expected statistics grouped by pitch type."""
    response = requests.get(
        f'https://statsapi.mlb.com/api/v1/people/{person_id}/stats/grouped',
        params={
            'stats': 'EXPECTED_STATISTICS',
            'group': group,
            'season': season,
            'groupBy': 'pitch_type'
        }
    )
    # Parse response to extract xBA, xSLG, xWOBA by pitch type
    return parse_expected_stats(response.json())

# Get stats for all unique players including expected stats
for pitcher_id in unique_pitchers:
    pitcher_stats = get_player_stats(pitcher_id, 'pitching', season)
    pitcher_matchups = get_matchup_history(pitcher_id, season)
    pitcher_expected = get_expected_stats_by_pitch(pitcher_id, 'pitching', season)
    # pitcher_expected contains: {'FF': {'xBA': .245, 'xSLG': .380}, 'SL': {...}, ...}

for batter_id in unique_batters:
    batter_stats = get_player_stats(batter_id, 'hitting', season)
    batter_matchups = get_matchup_history(batter_id, season)
    batter_expected = get_expected_stats_by_pitch(batter_id, 'hitting', season)
    # batter_expected contains stats vs each pitch type faced
```

6. Merge Data Sources into Flat Dataset

Create a denormalized dataset where each row represents a single pitch with all relevant contextual features. This format is optimal for most machine learning frameworks.

7. Feature Engineering

Sequence Features:

- Create lagged variables for previous 1-3 pitches
- Calculate running statistics within games and at-bats
- Encode pitch type sequences as categorical features

Interaction Features:

- Count-specific pitcher tendencies
- Batter performance vs specific pitch types
- Situational success rates
- Expected statistics interactions with count and weather

8. Model Training Considerations

Model Architecture Options:

- **LSTM/GRU**: Good for capturing sequential patterns in pitch sequences
- **Transformer Models**: Excellent for complex sequence dependencies
- **Gradient Boosting**: Strong baseline with good interpretability (XGBoost, LightGBM)
- **Random Forest**: Robust for feature importance analysis

Training Strategy:

- Use temporal splits (e.g., train on early season, test on late season)
- Consider pitcher-specific models for high-volume starters
- Implement cross-validation respecting game boundaries

Section 8: Additional Resources

Useful Lookup Endpoints

Reference Data:

- `/api/v1/teams` - Complete team information and IDs
- `/api/v1/gameTypes` - Game type codes and descriptions
- `/api/v1/statTypes` - Available statistical categories
- `/api/v1/pitchTypes` - Pitch type codes and descriptions
- `/api/v1/eventTypes` - Play event types and outcomes

API Configuration

Base URL: `https://statsapi.mlb.com`

Example Full Request:

```
https://statsapi.mlb.com/api/v1.1/game/717614/feed/live
```

Response Format: All endpoints return JSON format

Data Storage Recommendations

For Development:

- CSV files for small datasets and prototyping
- Parquet files for larger datasets with complex schemas
- SQLite for local development with relational queries

For Production:

- PostgreSQL or MySQL for structured relational data
- Apache Parquet on cloud storage (S3, GCS) for analytics workloads
- Time-series databases (InfluxDB) for real-time applications

This comprehensive guide provides all the necessary components to build a robust pitch sequence prediction dataset using the MLB Stats API. The key to success lies in systematic data collection, thorough feature engineering, and careful handling of the sequential nature of baseball data, enhanced with expected statistics and environmental factors for maximum predictive power.